The literature gap, drawn

four Lithuanian-related articles, what they leave open, and what this project did

1 WHAT THE FOUR ARTICLES DID

Kapočiūtė-Dzikienė et al. (2019)

Compared naive Bayes with LSTM and CNN on Lithuanian comments (3 classes).
Result: NB 0.735 beat CNN 0.706.

Ulčar & Robnik-Šikonja (2021)

Built LitLat BERT for Lithuanian, Latvian and English.
Result: corpus and vocabulary size matter.

Vileikytė et al. (2024)

Fine-tuned multilingual BERT and T5 on Lithuanian online reviews.
Result: high accuracy on clear reviews.

Nakvosas et al. (2024)

Released the first open Llama2-based Lithuanian models (7 B and 13 B parameters).
Result: new models and benchmarks.

2 WHAT EACH ONE LEAVES OPEN

Problem / limit

Showed that networks without pre-training lose on small data, but did not test a pre-trained Transformer or measure cost.

Problem / limit

Focus is building the model; behaviour with few labelled examples and cost on ordinary hardware are not the question.

Problem / limit

Reports accuracy; training time, latency, size and a comparison with cheap classical models are not the focus.

Problem / limit

Needs large hardware; says little about what a small team on a laptop should choose or how many labels it needs.

3 THE GAP THEY SHARE

GAP 1

Nobody measures quality, cost and data size together under one protocol on ordinary hardware.

GAP 2

How many labels does a pre-trained model need compared with a classical one?
(small data question)

GAP 3

Ready-made models vs a model fine-tuned on your own data: seldom compared fairly.

4 WHAT WE DID

1. Ask 5 research questions
2. Take one dataset: SST-2
3. Build 10 models, 4 families
4. Same laptop, same data, same rules
5. Measure accuracy, time, speed, size
6. Add data-size curve and statistics
7. Recommend what to use when

THEORY IN ONE LINE (explained on page 2)

A network trained from scratch must learn the language AND the task from the few labelled examples it has. A pre-trained Transformer already knows the language, so it only has to learn the task.
With 500 labelled sentences about 29% of the words in our test set were never seen in training: a model trained from scratch has no idea what they mean.

Why does small data hurt? theory first, then our own numbers

IN PLAIN WORDS

A classifier learns from examples. If it has only a few examples it cannot see most words, and a big model can simply memorise the few sentences it has been shown. It then looks perfect on its training data and fails on new sentences. A simple model with few parts (naive Bayes) cannot memorise much, so it is more careful and often wins when data are scarce.

THE THEORY (BIAS AND VARIANCE)

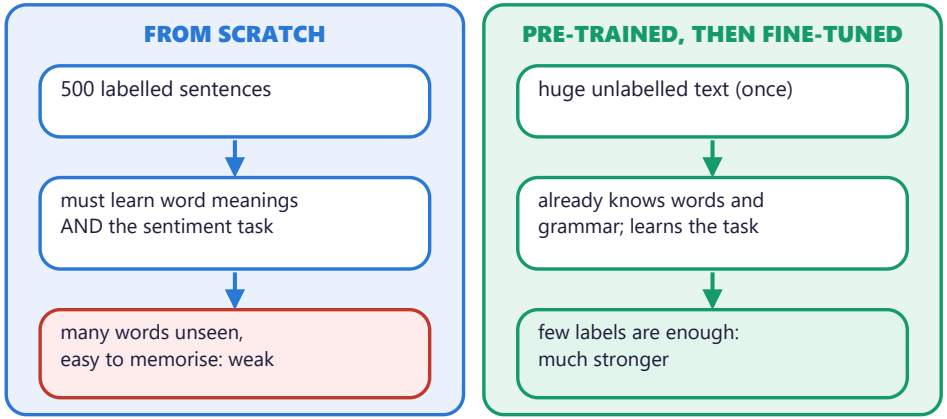
The error of a model on new data has two parts:

$$\text{expected error} \approx \text{bias}^2 + \text{variance} + \text{noise}$$

Bias is error from a model that is too simple. **Variance** is error from a model that is too flexible: it changes a lot depending on which few examples it saw. A network with millions of free numbers has low bias but **high variance**, and variance only falls when the number of training examples grows. A naive Bayes model has few numbers and strong assumptions: higher bias, **low variance**, so it does better with little data. This is exactly the 2019 Lithuanian result (naive Bayes 0.735 against CNN 0.706).

WHAT PRE-TRAINING CHANGES

Pre-training spends the "learning words and grammar" cost once, on huge unlabelled text. Fine-tuning then only has to learn the sentiment task, so the variance problem shrinks even with few labels. That is why the pre-trained Transformer in the Lithuanian papers (LitLat BERT, mBERT, T5) works where a scratch network does not.

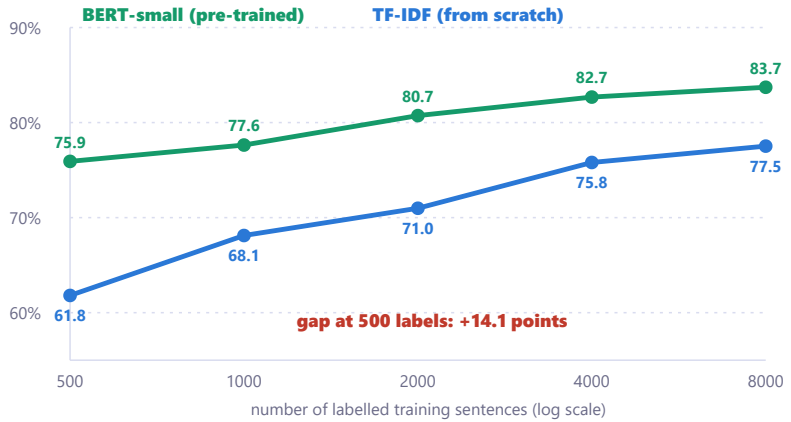


Our data, part 1: unseen words

500 labels	**29.5% unseen**	1,812 known words
2000 labels	**17.4% unseen**	4,531 known words
8000 labels	**9.1% unseen**	9,101 known words
all 65,349 labels	**5.4% unseen**	13,660 known words

Share of the running words in our 872 test sentences that never occur in a random training subset (SST-2, seed 42). Computed for this document.

Our data, part 2: accuracy against the number of labels



What we did, step by step

with one real sentence followed through the experiment

THE RUNNING EXAMPLE (TRUE LABEL: NEGATIVE)

“a great ensemble cast ca n't lift this heartfelt enterprise out of the familiar .”

A word-counting model sees *great* and *heartfelt* (both positive) and misses that “can't lift … out of” turns the sentence negative. This is the kind of sentence a good model must understand.

1 Find the gap

Read the four articles and the Google Scholar and Web of Science results; note what each leaves open.

Gap: no joint measure of quality, cost and data size.

2 Ask five questions

RQ1 accuracy, RQ2 cost, RQ3 data size, RQ4 ready-made models, RQ5 negation and contrast.

Each question has one measurement.

3 Choose the data

SST-2 movie reviews: 65,349 training, 2,000 development, 872 test sentences.

Our example is one of the 872 test sentences.

4 Build ten models

TF-IDF with 3 classifiers, a BiLSTM, 3 Transformers we fine-tune, 3 ready-made ones.

Four families, one fair set-up.

5 Same rules for all

One laptop CPU (no GPU), one data split, one random seed, the same test set.

So differences come from the model, not the set-up.

6 Measure quality

Accuracy, macro-F1 and accuracy on negation (184) and contrast (130) sentences.

See the table: 4 of 10 models get the example right.

7 Measure cost and data size

Training time, speed per sentence, model size, and a learning curve from 500 to 8,000 labels.

500 labels: BERT-small 75.9% vs TF-IDF 61.8%.

8 Check and conclude

Confidence intervals and McNemar's test; then practical advice for what to use when.

Fine-tuned DistilBERT beats TF-IDF, $p < 0.001$.

Step 6 in practice: the ten models on the example sentence

Model	Predicts	Result
TF-IDF + naive Bayes	positive	✗ wrong
TF-IDF + logistic regression	positive	✗ wrong
TF-IDF + linear SVM	positive	✗ wrong
BiLSTM (from scratch)	positive	✗ wrong
BERT-tiny fine-tuned (8,000 labels)	positive	✗ wrong
BERT-small fine-tuned (8,000 labels)	positive	✗ wrong
DistilBERT fine-tuned (8,000 labels)	negative	✓ correct
DistilBERT-SST2 (ready-made)	negative	✓ correct
Twitter-RoBERTa (ready-made)	negative	✓ correct
Zero-shot NLI (no labels)	negative	✓ correct

What this shows

WHY THE ANSWERS DIFFER

All three TF-IDF models and the BiLSTM, which learn only from our labelled sentences, call it positive. The three larger Transformers and the zero-shot model get it right: from pre-training they already know how “can't lift” and “familiar” behave in English. The two small fine-tuned models (BERT-tiny, BERT-small) have less capacity to use that knowledge and also miss it.

THE SAME STORY IN NUMBERS

Accuracy on negation sentences: TF-IDF + SVM 82.9% overall but 74.5% on negation; DistilBERT-SST2 91.1% overall and 87.0% on negation. Bag-of-words models lose 8 to 9 points on negation, large Transformers 2 to 4.

THE PRICE

The better models are 20 to 2,000 times slower per sentence and much larger. The gap we filled is exactly this trade-off: quality, cost and data size, measured together.

How to write and verify the gap for the literature review and for the viva

Three sentences that state the gap

TEMPLATE WITH OUR CONTENT

1. **What the literature shows:** studies of Lithuanian sentiment classification show that pre-trained models (LitLat BERT, multilingual BERT and T5, Lithuanian Llama2) work, and that neural networks without pre-training do not beat classical methods on small data (naive Bayes 0.735 against CNN 0.706).
2. **What it lacks:** these studies report accuracy; they do not measure training and inference cost, model size, or how many labelled examples a pre-trained model needs, and they seldom compare ready-made models with a fine-tuned in-domain model under one protocol.
3. **What we do:** this project compares ten models of four families under identical CPU-only conditions, measuring quality, cost, data efficiency and robustness.

Why this is a real gap and not just a different question

A reader who must choose a model for a real task needs three numbers together: how good, how expensive, how much labelled data. The articles give the first. We add the other two on the same data.

How to find it yourself in Google Scholar and Web of Science

1 Search	`("transformer" OR "BERT" OR "self-attention") AND ("sentiment analysis" OR "text classification")`; add `AND ("Lithuanian" OR "Baltic")` for the local part.
2 Filter	Google Scholar: since 2019, sort by relevance. Web of Science: Document type Article, category Computer Science, Artificial Intelligence.
3 Screen	Read titles and abstracts; keep papers that report a classifier result on a text task.
4 Mine the gap sentences	In each paper's introduction and conclusion look for “however”, “few studies”, “limited”, “future work”. Copy each into a table: author, year, what they did, what they left out.
5 Find the pattern	When several papers leave out the same thing, that is the gap.
6 Track citations	Use “Cited by” for newer work and the reference list for older work.
7 Verify	Open the original of every article you cite and check the numbers (for example 0.735 and 0.706) before they go into the paper.

IMPORTANT

The “what each article leaves open” boxes on page 1 are our reading of the articles as summarised in our literature review. Confirm each one against the original text before presenting it as a fact.